

number_theory — User Guide

`number_theory` is a small, dependency-free tool for **GCD** and **LCM** (and the extended Euclidean algorithm). The math ships as a **compiled binary**, so you install and use it without any readable source for the algorithms.

You can use it two ways:

- as a **command-line tool** (`number-theory ...`), or
- as a **Python library** (`import number_theory`).

1. Requirements

The prebuilt release wheel targets:

Operating system	Windows, 64-bit
Python	CPython 3.14 , 64-bit

On any other OS or Python version, install by **building from source** (see [1.3](#)) — you'll need a C compiler.

2. Installation

Tip: installing into a **virtual environment** keeps your global Python clean: `powershell python -m venv .venv`
`.\.venv\Scripts\Activate.ps1 # Windows PowerShell`

2.1 From the GitHub Release (recommended)

```
pip install https://github.com/Kuantor/number_theory/releases/download/v0.1.0/number_theory-0.1.0-cp314-cp314-win_amd64.whl
```

Or download the `.whl` from the [releases page](#) and install the file:

```
pip install .\number_theory-0.1.0-cp314-cp314-win_amd64.whl
```

2.2 Verify the install

```
number-theory --version          # -> number_theory 0.1.0  
python -c "import number_theory as nt; print(nt.gcd(12, 18))"  # -> 6
```

2.3 Build from source

Needs a C compiler (MSVC Build Tools on Windows; gcc/clang on Linux/macOS).

```
git clone https://github.com/Kuantor/number_theory
cd number_theory
pip install build
python -m build --wheel
pip install (Get-ChildItem dist\*.whl)    # PowerShell
# on macOS/Linux: pip install dist/*.whl
```

3. Command-line usage

```
number-theory <command> [numbers...]
```

Command	Meaning	Example	Output
<code>gcd N [N ...]</code>	Greatest common divisor	<code>number-theory gcd 12 18 24</code>	<code>6</code>
<code>lcm N [N ...]</code>	Least common multiple	<code>number-theory lcm 4 6 8</code>	<code>24</code>
<code>egcd A B</code>	Extended gcd: prints <code>g x y</code> with $A*x + B*y = g$	<code>number-theory egcd 240 46</code>	<code>2 -9 47</code>
<code>--version</code>	Print the version	<code>number-theory --version</code>	<code>number_theory 0.1.0</code>
<code>-h / --help</code>	Show help (works per command too)	<code>number-theory gcd -h</code>	usage text

Notes:

- `gcd` and `lcm` accept **two or more** integers.
- Negative integers are accepted; results are non-negative.
- If you run `number-theory` with no command it prints a usage error.

You can also run it without the console script:

```
python -m number_theory gcd 1071 462    # -> 21
```

4. Python API

```
import number_theory as nt
```

Function	Returns	Description
<code>gcd(a, b)</code>	<code>int</code>	Greatest common divisor. Non-negative. <code>gcd(0, 0) == 0</code> .
<code>lcm(a, b)</code>	<code>int</code>	Least common multiple. Non-negative. <code>lcm(x, 0) == 0</code> .
<code>extended_gcd(a, b)</code>	<code>(g, x, y)</code>	<code>g == gcd(a, b)</code> and <code>a*x + b*y == g</code> .
<code>gcd_many(*nums)</code>	<code>int</code>	GCD of two or more integers.
<code>lcm_many(*nums)</code>	<code>int</code>	LCM of two or more integers.
<code>nt.__version__</code>	<code>str</code>	Installed version, e.g. <code>"0.1.0"</code> .

Examples

```

nt.gcd(12, 18)          # 6
nt.gcd(-12, 18)         # 6      (sign-insensitive)
nt.lcm(4, 6)            # 12
nt.lcm(0, 5)            # 0

nt.gcd_many(12, 18, 24) # 6
nt.lcm_many(4, 6, 8)    # 24

g, x, y = nt.extended_gcd(240, 46)
# g == 2, and 240*x + 46*y == 2

# Arbitrary precision – works for huge numbers:
nt.gcd(2**512 * 3, 2**300 * 9) # exact, no overflow

```

Input rules

- Inputs must be Python `int`. Passing a `float`, `str`, `None`, or `bool` raises `TypeError`.
- `gcd_many()` / `lcm_many()` with **no** arguments raise `ValueError`.

5. Troubleshooting

ModuleNotFoundError: No module named 'number_theory._core' You're almost certainly running Python from **inside a clone of the repo**, so Python imports the local source folder (which has no compiled binary) instead of the installed package. Fix: run from a different directory, e.g.

```

cd ~
python -c "import number_theory as nt; print(nt.gcd(12, 18))"

```

'number-theory' is not recognized as an internal or external command The install worked — `pip` just placed the `number-theory.exe` launcher in your Python's `Scripts` folder, which isn't on your `PATH`. Three ways to deal with it:

Option A — run the module form (no PATH change needed):

```
python -m number_theory gcd 12 18
```

Option B — call the launcher by its full path. Find the folder with:

```
python -c "import sysconfig; print(sysconfig.get_path('scripts'))"
```

then run e.g. `& "<that folder>\number-theory.exe" --version`.

Option C — add that folder to your user PATH permanently (PowerShell):

```
$scripts = python -c "import sysconfig; print(sysconfig.get_path('scripts'))"
$u = [Environment]::GetEnvironmentVariable("Path", "User")
if ($u -notlike "$scripts*") {
    [Environment]::SetEnvironmentVariable("Path", "$u;$scripts", "User")
    "Added - open a NEW terminal for it to take effect."
} else { "Already on PATH." }
```

Then open a **new** terminal and `number-theory --version` will work.

Avoid the naive `setx PATH "%PATH%;..."` in cmd: `setx` truncates `PATH` at 1024 characters and folds your combined system+user `PATH` into the user variable, which can corrupt it. The PowerShell snippet above edits only the user scope and is safe.

If you're working inside a **virtual environment**, activating it (`.\.venv\Scripts\Activate.ps1`) already puts the launcher on `PATH` — no manual step needed.

pip says the wheel is "**not a supported wheel on this platform**" The release wheel is for Windows x64 + Python 3.14 only. On a different platform or Python version, build from source (see 2.3).

6. Uninstall

```
pip uninstall number_theory
```

7. Updates & links

- Releases: https://github.com/Kuantor/number_theory/releases
- Source & issues: https://github.com/Kuantor/number_theory
- Roadmap (upcoming: primality, factorization, totient, modular arithmetic):
<https://github.com/orgs/Kuantor/projects/10>

This guide covers **v0.1.0**, whose scope is GCD / LCM only.